

GigaCode CLI для реверсивной аналитики кода

Подробная версия для обсуждения архитектуры, флоу и конфигурации рабочего контура

Краткое резюме

Проблема Знание о реализации бизнес-фич хранится в головах разработчиков, в устаревшем Confluence и закрытых Jira-тикетах. Разобраться в фиче без помощи конкретного человека занимает 2–3 дня.	Предлагаемое решение GigaCode CLI анализирует код, Jira и Confluence по одной бизнес-фиче и создаёт задокументированный артефакт рядом с кодом в репозитории — с явными источниками и списком открытых вопросов.
Что система делает Ищет и картирует код, извлекает контекст из Jira и Confluence, пишет черновик по шаблону, проверяет каждое утверждение на наличие источника, помечает противоречия и пробелы.	Что остаётся за аналитиком Выбрать фичу, ответить на уточняющие вопросы системы, принять решения по спорным местам и одобрить результат перед сохранением в репозиторий.
Ожидаемый результат пилота Время до первого черновика — менее 15 минут. Доля подтверждённых утверждений — более 80%. Объём ручных правок — менее 30% текста.	Стартовый объём Пилот на 2–3 фичах средней сложности. Пять агентов, один общий свод правил, два контрольных блока. Без избыточной автоматизации.

Содержание

1. Суть идеи и проблема, которую решаем
2. Ключевые принципы
3. Полный рабочий флоу (схема)
4. Архитектура: пять агентов, свод правил, контрольные блоки

Состав решения v1: skill, hooks, утилиты и что нужно настроить

5. Управление контекстом и стратегия разбивки
6. Инструкции агентов
7. Структура артефактов в репозитории
8. Сквозной пример
9. Что не автоматизируется
10. Ограничения первой версии
11. Рекомендации по пилоту и показатели успеха

1. Суть идеи и проблема, которую решаем

Текущая ситуация

Документация по бизнес-фичам существует в трёх разрозненных местах:

- **Код** — единственный источник текущей реализации, но без контекста непонятен
- **Jira** — описание требований, однако часто устаревает после релиза
- **Confluence** — архитектурные описания, поддерживаются нерегулярно
- **Знания в головах** — критичные детали хранятся у конкретных разработчиков

Результат: новому сотруднику или аналитику нужно 2–3 дня, чтобы разобраться в одной фиче.

Предлагаемое решение

Использовать GigaCode CLI как оркестратор обратного анализа:

1. Пользователь выбирает конкретную бизнес-фичу — не весь сервис целиком
2. Система анализирует код, Jira и Confluence, восстанавливает фактическую реализацию
3. Результат сохраняется в репозитории рядом с кодом как инженерный артефакт
4. Документация строится по строгому шаблону с явными источниками всех утверждений
5. Всё неподтверждённое выносится в предположения или открытые вопросы

Ключевой принцип: итоговая документация — не свободное описание системы, а доказуемый артефакт по конкретной функции с явными источниками, открытыми вопросами и контролем границ анализа.

Цели решения

- Снизить зависимость от ситуации, когда знание живёт только в головах: вместо поиска коллеги аналитик открывает docs/features/<название>/overview.md
- Сделать обратный анализ воспроизводимой процедурой: любой сотрудник запускает анализ по тому же алгоритму
- Получать документацию по единому шаблону, пригодную для дальнейшей поддержки
- Использовать код как первичный источник: если код и Jira расходятся, код — это правда, расхождение фиксируется отдельно

2. Ключевые принципы

Доказуемость каждого утверждения

Каждое утверждение в документации должно иметь явный источник:

- **код** — подтверждено анализом исходников
- **jira** — взято из тикета или описания требований
- **confluence** — найдено в архитектурной документации
- **пользователь** — сообщено при запуске анализа

Правило: если факт не подтверждён ни одним источником, он переводится в предположения или открытые вопросы, а не подаётся как истина.

Приоритет источников при расхождениях

Источник	Статус	Действие при расхождении
Код	Источник текущей реализации	Код считается правдой
Jira	Требования (могут устареть)	Фиксируется как расхождение в вопросах.md
Confluence	Пояснения (могут быть неточными)	Используется как контекст, но не как истина

Прочие принципы

- **Единица анализа** — одна бизнес-функция, а не весь сервис. Сначала определяется область охвата, затем строится карта кода, только потом создаётся документация
- **Документация как инженерный артефакт.** Хранится в том же репозитории, что и код, проходит проверку через запрос на слияние, история отслеживается через журнал изменений
- **При неоднозначности — спрашивать.** Лучше задать лишний вопрос пользователю, чем додумать неправильно

3. Полный рабочий флоу

Цветовая кодировка: синий = агент, оранжевый = человек, красный = контрольный блок. Весь сценарий запускается внутри общего skill reverse-analysis; контрольные блоки в схеме соответствуют preflight и output validation hooks, а также внутренней контрольной точке качества.

ЧЕЛОВЕК	Шаг 1: запуск команды gigacode reverse-analysis --feature "Блокировка карты" --jira TKT-1234 или gigacode > Проведи reverse-analysis функции "Блокировка карты", тикет в JIRA <ссылка>
КОНТРОЛЬ	Предварительная проверка: обязательные вводные Проверяет: название функции, наличие Jira/Confluence (опционально), целевой каталог, шаблон документа, текущая ветка. Если чего-то не хватает — возвращает конкретный список вопросов, к коду не переходит.
АГЕНТ	Шаг 2: агент сбора контекста Извлекает данные из Jira и Confluence через коннектор Atlassian. Формирует краткое описание области охвата: что анализируем, что исключено, известные пробелы. Если внешний контекст не указан — фиксирует, что анализ будет вестись только по коду. <i>Результат: область_охвата.json</i>
АГЕНТ	Шаг 3: агент картирования кода Использует heromix и graphify для построения графа зависимостей. Находит точки входа, основные модули, интеграции (API, очереди, события), границы функции. Если цепочка не прослеживается — помечает пробел. <i>Результат: карта_кода.json + список_файлов.txt</i>
ЧЕЛОВЕК	Шаг 4: подтверждение области охвата Система показывает: что включено в анализ, что исключено, какие пробелы уже известны. Пользователь подтверждает или корректирует границы.
АГЕНТ	Шаг 5: создание рабочей ветки Создаётся ветка по правилу именования, например: docs/reverse-analysis/card-block-sms.
АГЕНТ	Шаг 6: агент документирования Пишет документацию строго по шаблону. Каждое существенное

	утверждение помечается источником: [код], [jira], [confluence], [пользователь]. Неподтверждённые тезисы выносятся в предположения или вопросы. <i>Результат: черновик/*.md</i>
АГЕНТ	Шаг 7: агент проверки доказательств Проверяет каждое утверждение: подтверждено кодом, подтверждено внешней документацией, противоречие, или доказательств нет. Строит матрицу доказательств. <i>Результат: матрица_доказательств.json + пробелы.json</i>
КОНТРОЛЬ	Контрольная точка качества Если охват доказательств < 70% И критических пробелов > 0 — возврат к агенту картирования для более глубокого поиска. Если найдены противоречия — вопрос к пользователю: «Что считать правдой?» Если всё в порядке — переход к агенту ревью.
АГЕНТ	Шаг 8: агент ревью Финальная проверка: полнота по шаблону, согласованность терминов, орфография, наличие открытых вопросов. Новых фактов не добавляет. <i>Результат: итоговый_черновик/ + открытые_вопросы.md</i>
КОНТРОЛЬ	Проверка выходного артефакта Проверяет: файлы в нужной папке, все секции шаблона заполнены, блок открытых вопросов присутствует, нет пустых разделов и маркеров незавершённости (ЗАДАЧА, ПОД ВОПРОСОМ). При провале — возврат к агенту ревью.
ЧЕЛОВЕК	Шаг 9: финальная проверка и фиксация Пользователь получает черновик: документация, предположения, открытые вопросы, спорные места. Вносит правки при необходимости, создаёт запрос на слияние для проверки коллегами.

4. Архитектура: пять агентов, общий skill, hooks и контрольные блоки

Компактное ядро выбрано сознательно: на первом этапе системе важнее предсказуемость и поддерживаемость, чем максимальная гранулярность ролей. Пять агентов дают достаточное разделение ответственности без роста стоимости сопровождения и без расползания промптов.

Почему именно 5 агентов, а не 8–10

На старте важнее устойчивый контур, чем максимальная детализация ролей. Каждая новая роль — это отдельный prompt, отдельный контракт входов и выходов, отдельный источник ошибок и дополнительная стоимость поддержки. Пять агентов покрывают полный цикл: сбор контекста, картирование, написание, проверка доказательств и финальное ревью. Более дробная схема имеет смысл только после пилота, когда появятся реальные узкие места: например, отдельный consistency-agent или отдельный hero-operations-agent. Выбор компактного ядра — архитектурное решение первой версии, а не ограничение платформы.

Роль	Зона ответственности	Результат
Агент сбора контекста	Собирает постановку, проверяет обязательные вводные, задаёт уточняющие вопросы, фиксирует ограничения анализа	область_охвата.json, список недостающих данных
Агент картирования кода	Строит карту кода: точки входа, модули, интеграции, события, цепочки вызовов, границы функции	карта_кода.json, список релевантных файлов
Агент документирования	Пишет документы по шаблону, расставляет источники, раскладывает файлы по структуре каталогов	черновик/*.md
Агент проверки доказательств	Проверяет, какие утверждения подтверждены, где пробелы, где	матрица_доказательств.json, пробелы.json

	противоречия между источниками	
Агент ревью	Финальная проверка: полнота шаблона, согласованность терминов, орфография, наличие открытых вопросов	итоговый черновик, открытые_вопросы.md

Состав решения v1: skill, hooks, утилиты и что нужно настроить

Помимо пяти агентов, рабочий контур включает единый skill, два hook-блока и набор внешних утилит. Это не “дополнительные украшения”, а части операционного контура, которые удерживают систему в дисциплине и не дают ей преждевременно переходить к генерации текста.

1 общий skill: reverse-analysis

Skill — это единая рамка поведения, которая активируется при запуске сценария reverse-analysis.

Он содержит: правила по источникам, шаблон документа, требования к постановке вопросов, правила работы с границами анализа и запрет на домысливание фактов.

Skill нужен для того, чтобы все агенты работали в одном режиме и не начинали “писать каждый по-своему”.

2 обязательных hook-а

Preflight hook запускается до анализа кода и проверяет обязательные вводные: название функции, наличие или явное отсутствие Jira/Confluence, целевой каталог, шаблон документа, допустимую ветку.

Output validation hook запускается после генерации черновика и проверяет структуру результата: обязательные секции, наличие блока открытых вопросов, отсутствие пустых разделов и маркеров незавершённости.

Hooks важны не меньше агентов: агенты отвечают за рассуждение, а hooks — за дисциплину процесса.

Ключевые утилиты

Atlassian MCP — получение контекста из Jira и Confluence.

repomix — структура репозитория и укрупнённая карта модулей.

graphify — граф зависимостей и подграф по конкретной функции.

Git / naming rules — создание ветки, сохранение результата и проверяемый жизненный цикл артефакта в репозитории.

Что нужно настроить для v1

GigaCode CLI с отдельным режимом reverse-analysis.
Подключённый Atlassian MCP с доступом к Jira и Confluence.
Установленные и проверенные repomix и graphify.
Шаблон документа функции, правила веток и контракты агентов в репозитории.
Один общий skill и два hook-а как обязательная часть конфигурации.

Общий свод правил

Единая рамка поведения для всего сценария. Не отдельный агент, а набор инвариантов, которые активируются при запуске обратного анализа.

- Контрольный список вводных (обязательные данные для старта)
- Правила источников: код / jira / confluence / пользователь
- Запрет на домысливание фактов: неподтверждённое уходит в предположения или вопросы
- Требования к итоговому шаблону: обязательные секции, форматирование
- Правила постановки вопросов пользователю

Предварительная проверка

Срабатывает перед началом анализа. Если обязательных данных нет — не переходит к коду, возвращает список вопросов.

- Название функции указано
- Наличие или отсутствие Jira/Confluence зафиксировано
- Целевой каталог существует
- Шаблон документа доступен
- Текущая ветка соответствует правилам именования

Проверка выходного артефакта

Срабатывает перед показом результата пользователю. Дисциплинирует итоговый файл.

- Файлы созданы в нужной папке
- Все обязательные секции шаблона заполнены
- Блок открытых вопросов присутствует и не пуст
- Нет пустых разделов и маркеров незавершённости

5. Управление контекстом и стратегия разбивки

Главный принцип этого раздела: система не должна пытаться “загрузить весь репозиторий и как-нибудь разобраться”. Сначала строится карта, затем выделяется подграф, и только после этого в контекст попадает код релевантных файлов.

Почему нельзя просто загрузить всё сразу

Составляющая	Размер	Примечание
Кодовая база	~160 000 токенов	40 000 строк × 4 токена
Jira + Confluence по одной фиче	20 000–50 000 токенов	зависит от полноты документации
Инструкции агентов	5 000–10 000 токенов	system prompt
Промежуточные артефакты	5 000–10 000 токенов	передача между агентами
Буфер на ответ	5 000–10 000 токенов	генерируемая документация
Реальный лимит на ввод	~200 000 токенов	при окне 256 000

Вывод: если фича затрагивает более 30% кодовой базы (~12 000 строк), загрузить всё одновременно не получится. Нужна разбивка.

Трёхэтапная стратегия разбивки

Этап 1: граф вместо кода

- Агент картирования использует herotix для получения структуры репозитория (список файлов, модулей, зависимостей)
- Использует graphify для построения графа зависимостей (кто кого вызывает, кто кого импортирует)
- Находит точки входа (контроллеры, обработчики событий, планировщики)
- Выделяет минимальный подграф для конкретной функции

Граф занимает 1 000–2 000 токенов. Весь код не загружается.

Этап 2: только релевантные файлы

После построения графа загружаются только файлы из минимального подграфа. Для типичной фичи это 5–15 файлов вместо всего репозитория.

Пример: фича «Блокировка карты по СМС» затрагивает 6 файлов $\approx 1\,500$ строк $\approx 6\,000$ токенов вместо 160 000.

Этап 3: фильтрация Jira и Confluence

Агент сбора контекста не загружает весь тикет — только:

- Описание и критерии приёмки из Jira
- Последний комментарий с итогом, если задача завершена
- Только тот раздел Confluence, который относится к конкретной функции

Проекции между агентами

Каждый агент получает только свою часть данных:

Агент	Получает в контекст
Картирование кода	область_охвата + граф зависимостей (без полного кода)
Документирование	карта_кода + код релевантных файлов + краткое изложение Jira/Confluence
Проверка доказательств	черновик/*.md + карта_кода (для сверки утверждений с кодом)
Ревью	черновик/*.md + матрица_доказательств + шаблон

Правило: между агентами передаются краткие сводки и примечания к доказательствам, а не полная история переписки.

6. Инструкции агентов

Ниже приведены расширенные инструкции агентов. Для основной презентации достаточно описания ролей и артефактов, а этот раздел следует воспринимать как приложение к концепции и как заготовку для будущих рабочих prompt-файлов. Ниже приведены инструкции для каждого агента. Это НЕ финальные рабочие варианты, а краткие примеры.

Агент сбора контекста и области охвата

Роль: агент сбора контекста и области охвата

Ты отвечаешь за сбор минимально достаточного контекста для обратного анализа одной бизнес-функции. Твоя цель – сформировать проверенное описание области охвата: что анализируем, что исключено, какие пробелы уже известны.

Вводные данные

- Название функции (обязательно)
- Идентификатор Jira-задачи (опционально)
- Ссылка на Confluence (опционально)
- Ожидаемый результат (опционально)
- Целевой каталог документации (обязательно)

Задачи

1. Проверить вводные:

- Название функции достаточно конкретно (не «обработка платежей», а «блокировка карты по СМС»)
- Если указан Jira-идентификатор – получить задачу через коннектор
- Если указана Confluence – получить страницу через коннектор
- Если ни то ни другое не указано – явно зафиксировать: «анализ будет вестись только по коду»

2. Извлечь контекст:

- Из Jira: описание, критерии приёмки, последний комментарий с итогом
- Из Confluence: только раздел по конкретной функции (не всю страницу)
- Общий объём извлечённого текста – не более 10 000 токенов

3. Задать уточняющие вопросы, если:

- Границы функции неоднозначны
- Существует несколько возможных точек входа
- Упоминается интеграция с внешней системой, но детали непонятны

4. Сформировать область_охвата.json:

```
{
  "название_функции": "...",
  "бизнес_цель": "...",
  "внешний_контекст": {"jira": "...", "confluence": "..."},
  "включено": ["..."],
  "исключено": ["..."],
  "открытые_вопросы": ["..."]
}
```

Ключевые правила

- НЕ переходить к анализу кода без подтверждённой области охвата
- НЕ додумывать неявные требования
- Задавать конкретные вопросы, а не общие («Какова бизнес-цель?» — слишком широко)

Агент картирования кода

Роль: агент картирования кода

Ты строишь карту реализации функции: точки входа, ключевые модули, интеграции, события, границы. Работаешь с графом зависимостей и структурой файлов, а не с полным кодом.

Вводные данные

- область_охвата.json
- Структура репозитория (из `repomix`)
- Граф зависимостей (из `graphify`)

Задачи

1. Найти точки входа:

- API-эндпоинты (контроллеры, маршруты)
- Обработчики событий (подписчики очереди, вебхуки)
- Планировщики (`cron`, таймеры)
- Команды командной строки

2. Построить минимальный подграф:

- От точек входа проследить вызовы вниз
- Обязательно включать: прямые вызовы, изменения состояния, внешние интеграции
- Опционально включать: валидацию, обработку ошибок, аудит-лог
- НЕ включать: общие утилиты (`StringUtils`), инфраструктурный код (пулы соединений), другие функции

3. Определить границы:

- Где эта функция заканчивается и начинается другая?
- Если граница неоднозначна — добавить модуль в список спорных с обоснованием

4. Сформировать карту_кода.json:

```
{
  "точки_входа": [{"файл": "...", "метод": "...", "тип":
"api/событие/cron"}],
  "ключевые_модули": [{"файл": "...", "роль": "сервис/репозиторий/dto"}],
  "интеграции": [{"название": "...", "тип": "rest/очередь/бд"}],
  "события": [{"название": "...", "отправитель": "...", "получатели":
["..."]}],
  "спорные_модули": [{"файл": "...", "причина": "..."}],
  "пробелы": ["не удалось проследить вызов от X к Y"],
  "список_файлов": ["путь/к/файлу1", "..."]
}
```

Критерии включения

ОБЯЗАТЕЛЬНО включать:

- Прямые вызовы из точки входа
- Модули, изменяющие состояние системы
- Внешние интеграции

ОПЦИОНАЛЬНО (если есть явная связь):

- Валидация, обработка ошибок, аудит-лог

НЕ включать:

- Общие утилиты (StringUtils, DateUtils)
- Инфраструктурный код (retry-логика, пулы соединений)
- Другие функции, даже если косвенно связаны

Ключевые правила

- НЕ придумывать цепочки вызовов
- Если цепочка не прослеживается – помечать как пробел
- Если граница неоднозначна – спрашивать пользователя или помечать как спорный
- Держать список файлов в пределах 20 (цель: 10–15)

Агент документирования

Роль: агент документирования

Ты пишешь документацию строго по шаблону функции. Каждое существенное утверждение должно иметь явный источник: [код], [jira], [confluence] или [пользователь]. Неподтвержденные тезисы – только в предположения или вопросы, не в основной текст.

Вводные данные

- карта_кода.json
- Код релевантных файлов (из списка файлов)
- Краткое изложение Jira/Confluence (если доступно)
- шаблон_функции.md

Задачи

1. Создать обзор.md:

- Бизнес-цель функции [jira или пользователь]
- Краткое описание: что делает, зачем существует
- Ключевые пользователи / стейкхолдеры

2. Создать флоу.md:

- Пользовательский сценарий (шаги глазами пользователя)
- Системный сценарий (что происходит внутри на каждом шаге)
- Точки входа и переходы
- Для потока из кода – метка [код],
для задуманного поведения из документации – [confluence]

3. Создать интеграции.md:

- Внешние системы: название, тип (REST/SOAP/очередь),
направление (входящее/исходящее)
- Вызываемые API: эндпоинт, метод, назначение
- Публикуемые/получаемые события: топик, схема, обработчик

4. Создать данные.md:

- Ключевые сущности: доменные объекты, объекты передачи данных
- Таблицы базы данных и важные поля
- Схемы сообщений (если применимо)

5. Создать вопросы.md:

- Пробелы, обнаруженные при анализе
- Противоречия между источниками
- Предположения (например, «тайм-аут 30 с – предположение»)

Формат разметки источника

Правильно:

«Система блокирует карту, обновляя статус на ЗАБЛОКИРОВАНА
[код: КартаСервис.заблокироватьКарту()]. По требованиям должно
отправляться СМС-уведомление [jira: ТКТ-1234], но интеграция
не найдена в коде [пробел].»

Неправильно:

«Система блокирует карту и отправляет СМС.» (нет источников)

Ключевые правила

- Каждое утверждение – с меткой источника
- Если утверждение нельзя проверить – перенести в предположения
- НЕ придумывать поведение, не найденное в коде
- При расхождении Jira и кода – зафиксировать оба варианта и пометить противоречие

Агент проверки доказательств

Роль: агент проверки доказательств

Ты проверяешь каждое утверждение в документации. По каждому разделу определяешь: подтверждено кодом, подтверждено внешней документацией, есть противоречие, или доказательств нет.

Вводные данные

- черновик/*.md (все файлы документации)
- карта_кода.json
- Код релевантных файлов

Задачи

1. Для каждого утверждения в документации:

- Проверить наличие в коде
- Сверить с Jira/Confluence (если доступно)
- Выявить противоречия
- Отметить пробелы (утверждения без доказательств)

2. Сформировать матрицу_доказательств.json:

```
{
  "обзор.md": {
    "Бизнес-цель: блокировка украденных карт":
      {"подтверждено": ["jira"], "уверенность": "высокая"},
    "Используют клиенты мобильного приложения":
      {"подтверждено": ["confluence"], "уверенность": "средняя"}
  },
  "флоу.md": {
    "СМС-команда вызывает контроллер":
      {"подтверждено": ["код"], "уверенность": "высокая"},
    "Шлюз отвечает в течение 5 с":
      {"подтверждено": [], "уверенность": "нет",
        "примечание": "конфигурация тайм-аута не найдена"}
  }
}
```

3. Сформировать пробелы.json:

```
{
  "критические": [
    "Jira требует СМС-уведомление о блокировке, в коде отправителя нет",
    "Ожидаемый тайм-аут 30 с (Jira), фактический 60 с (код)"
  ],
  "незначительные": [
```

```
"Обработка ошибок не задокументирована",  
"Логика повтора запроса неясна"  
]  
}
```

Уровни уверенности

высокая — напрямую найдено в коде с точным соответствием
средняя — выведено из кода или подтверждено внешней документацией
низкая — упомянуто во внешней документации, но в коде не найдено
нет — доказательств не обнаружено

Пример:

Утверждение: «Карта блокируется после 3 неверных ПИН-кодов»

Код: `if (неверныхПопыток >= 5) { заблокировать(); }`

→ Противоречие: Jira говорит 3, код — 5

→ Уверенность: нет (несоответствие)

→ Действие: критический пробел

Ключевые правила

- НЕ оценивать самостоятельно, критично ли противоречие
- Фиксировать ВСЕ противоречия, пусть пользователь решает
- НЕ пытаться закрыть пробелы домыслами
- При неоднозначности доказательства — уверенность «низкая»

Агент ревью

Роль: агент ревью

Ты проводишь финальный контроль качества: проверяешь соответствие шаблону, согласованность терминологии, орфографию, наличие обязательных секций. Новых фактов не добавляешь.

Вводные данные

- черновик/*.md
- матрица_доказательств.json
- шаблон_функции.md

Задачи

1. Соответствие шаблону:

- Все обязательные разделы присутствуют
- Нет пустых разделов
- Нет маркеров незавершённости (ЗАДАЧА, ПОД ВОПРОСОМ, ЗАГЛУШКА)
- Файл вопросы.md существует и не пуст

2. Согласованность терминологии:

- Одна и та же сущность называется одинаково везде (не «карта» в одном месте и «платёжный инструмент» в другом)
- Технические термины соответствуют кодовой базе
- При необходимости – сформировать словарь терминов

3. Проверка стиля:

- Корректное форматирование Markdown
- Отсутствие нерабочих ссылок
- Метки источников проставлены там, где нужно
- Орфография и грамматика

4. Проверка полноты:

- Если матрица_доказательств содержит утверждения с уверенностью «нет» по критичным тезисам – поставить флаг для проверки пользователем
- Если есть критические пробелы – убедиться, что они в вопросы.md

5. Сформировать итог_ревью.json:

```
{
  "соответствие_шаблону": true/false,
  "проблемы_терминологии": ["Карта vs КартаDTO используются
непоследовательно"],
  "отсутствующие_разделы": [],
  "проблемы_стиля": ["Опечатка в флоу.md строка 42"],
```

```
    "требуется_проверки": ["Критический пробел: СМС-уведомление не  
реализовано"]  
}
```

Ключевые правила

- НЕ добавлять новые факты или утверждения
- НЕ закрывать пробелы вымышленной информацией
- НЕ менять метки источников
- ИСПРАВЛЯТЬ: опечатки, форматирование, несогласованность
- При обнаружении фактических проблем – ставить флаг для повторной проверки агентом доказательств

7. Структура артефактов в репозитории

Вся документация хранится в репозитории рядом с кодом. Это делает её частью нормального цикла проверки и версионирования.

Структура каталогов

```
docs/
├─ features/
│   └─ card-block-sms/
│       ├── обзор.md
│       ├── флоу.md
│       ├── интеграции.md
│       ├── данные.md
│       └─ вопросы.md
│   └─ cashback-accrual/
│       └─ ...
│   └─ ...
├─ templates/
│   └─ шаблон_функции.md
└─ rules/
    ├── контракты_агентов.md
    └─ правила_веток.md
```

Назначение файлов

Файл	Содержимое
обзор.md	Краткое описание функции и бизнес-цели
флоу.md	Пользовательский и системный сценарий, точки входа и переходы
интеграции.md	Внешние системы, API, очереди, события
данные.md	Сущности, объекты передачи данных, сообщения, таблицы, важные поля
вопросы.md	Открытые вопросы, пробелы, противоречия, предположения
шаблон_функции.md	Обязательный шаблон документа с описанием всех секций
контракты_агентов.m	Роли, границы, правила обмена артефактами между

d	агентами
правила_веток.md	Правила именования веток и коммитов для документации

8. Сквозной пример: фича «Блокировка карты по СМС»

Ниже приведён демонстрационный сценарий. Он нужен, чтобы показать работу контура на конкретной функции, а не чтобы ограничить решение банковским доменом.

Исходные данные

Параметр	Значение
Функция	Блокировка карты по СМС
Jira	TKT-1234
Confluence	https://wiki/sms-commands
Команда	<code>gigacode reverse-analysis --feature "Блокировка карты по СМС" --jira TKT-1234</code>

Шаг 1: агент сбора контекста

Извлекает из Jira: «Клиент должен иметь возможность заблокировать карту, отправив СМС с текстом *BLOCK на номер 900». Из Confluence — список поддерживаемых СМС-команд, требование отправки СМС-подтверждения клиенту. Задаёт уточняющий вопрос: «Включать ли проверку антифрода перед блокировкой?». Пользователь отвечает: «Нет, это отдельная функция».

Результат: область_охвата.json с включением [СМС-шлюз, обновление статуса карты], исключением [антифрод].

Шаг 2: агент картирования кода

Строит граф зависимостей через herotix и graphify, находит точку входа. Прослеживает цепочку:

1. КонтроллерСМС.обработатьБлокировку() → СервисКарт.заблокироватьКарту()
2. СервисКарт → РепозиторийКарт.обновитьСтатус()
3. СервисКарт → ИздательСобытий.опубликовать(КартаЗаблокирована)
4. СервисУведомлений.приКартеЗаблокирована() → отправляет push

Не находит: СМС-уведомление клиенту (Confluence говорит, что должно быть) — помечает как критический пробел.

Результат: карта_кода.json, 6 файлов в списке.

Шаг 3: пользователь подтверждает область охвата

Категория	Содержимое
Включено	СМС-команда *BLOCK, обновление статуса карты, push-уведомление
Исключено	Антифрод-проверка, другие СМС-команды
Известные пробелы	СМС-подтверждение клиенту — требуется по Confluence, не найдено в коде

Пользователь: «Подтверждаю, продолжайте».

Шаги 4–6: ветка, документирование, проверка доказательств

Создаётся ветка: docs/reverse-analysis/card-block-sms.

Агент документирования создаёт:

- **обзор.md**: «Функция позволяет клиенту заблокировать карту через СМС при утере [jira]. Статус обновляется через СервисКарт [код]»
- **флоу.md**: цепочка от СМС клиента до push-уведомления, шаг за шагом с метками источников
- **интеграции.md**: СМС-шлюз (входящий REST), Сервис уведомлений (исходящее событие)
- **вопросы.md**: СМС-подтверждение [критический пробел], обработка ошибок [незначительный], ограничение частоты запросов [незначительный]

Утверждение	Источник	Уверенность
СМС *BLOCK блокирует карту	код	высокая
Push-уведомление отправляется	код	высокая
СМС-подтверждение клиенту	—	нет (пробел)

Охват доказательств: 67% (2 из 3). Критических пробелов: 1.

Контрольная точка качества

Охват 67% < 70% и критических пробелов 1 > 0. Система спрашивает пользователя:

Найден критический пробел: СМС-подтверждение требуется по Confluence, в коде не найдено.

Варианты: 1) искать глубже, 2) принять пробел и продолжить, 3) уточнить у владельца функции.

Пользователь: «Принять пробел и продолжить».

Шаги 7–8: ревью и проверка выходного артефакта

- ✓ Все секции заполнены
- ✓ вопросы.md содержит критический пробел про СМС
- ✓ Терминология согласована (езде «карта», а не «платёжный инструмент»)
- ✓ Нет маркеров незавершённости

Итог: пользователь получает 4 файла документации, 1 критический и 2 незначительных вопроса, готовую ветку для запроса на слияние. Время выполнения: около 12 минут.

9. Что не автоматизируется

Система помогает аналитику, но не заменяет его. Чёткое понимание границ — обязательное условие для правильных ожиданий.

✓ СИСТЕМА ДЕЛАЕТ	✗ АНАЛИТИК ДЕЛАЕТ
Ищет код, связанный с функцией	Выбирает, какую функцию анализировать
Извлекает контекст из Jira и Confluence	Отвечает на уточняющие вопросы системы
Пишет черновик по шаблону с метками источников	Проверяет фактическую корректность черновика
Помечает пробелы и противоречия	Решает спорные места (код против Jira, граница функции)
Проверяет стиль и форматирование	Одобрывает результат и делает запрос на слияние
Создаёт рабочую ветку по правилу именования	Отвечает на вопросы при ревью коллег

Критические решения всегда за человеком

- Что делать при противоречии между источниками (код говорит одно, Jira — другое)
- Включать ли пограничный модуль в область охвата (например, аудит-лог)
- Продолжать ли анализ при низком охвате доказательств
- Как интерпретировать неоднозначный код (закомментированная логика, устаревшие ветки)

Правило: при любой неоднозначности система останавливается и спрашивает пользователя. Лучше задать лишний вопрос, чем зафиксировать неправильный факт.

10. Ограничения первой версии

Что не входит в первую итерацию. Расширение возможно по итогам пилота.

Функциональные ограничения

Ограничение	Пояснение
Только одна функция за раз	Если функции связаны, их анализируют последовательно
Нет авто-обновления	При изменении кода — запустить анализ заново вручную
Один репозиторий	Интеграции с другими сервисами помечаются как внешние, без их анализа
Нет диаграмм	Документация в Markdown. Схемы — вручную или в следующих версиях
Нет интеграции с мониторингом	Источники: только код, Jira, Confluence. Логи и метрики не анализируются

Технические ограничения

Ограничение	Пояснение
Максимальный охват: ~30% кодовой базы	При 40 000 строк — около 12 000 строк на функцию. При превышении система предупреждает и предлагает разбить функцию
Только Java (первая версия)	Агент картирования настроен на Spring-контроллеры, JPA-репозитории. Другие языки — после адаптации
Система не исправляет плохую документацию	Если Jira пуст или Confluence устарел — это зафиксировывается как пробел. Мусор на входе → пробел на выходе, не мусор на выходе

11. Рекомендации по пилоту и показатели успеха

Выбор функций для пилота

Не брать

- **Слишком простые** — накладные расходы системы не оправданы (задокументировать вручную за 15 минут)
- **Слишком сложные** — не войдут в контекст, потребуют слишком много ручных итераций
- **Устаревший код без документации** — система не восстановит намерения, результат будет полон пробелов

Идеальные кандидаты

- 3–5 модулей, 1–2 интеграции
- Есть хотя бы минимальная документация в Jira или Confluence
- Чётко выделенная бизнес-функция с понятными границами
- Реализована 1–2 года назад: код ещё читаем, документация не слишком устарела

Примеры хороших кандидатов

- Смена ПИН-кода карты через мобильное приложение
- Начисление процентов по вкладу в конце месяца
- Отправка push-уведомлений при изменении статуса заявки
- Блокировка карты по СМС-команде (как в примере выше)

Показатели успеха

Показатель	Целевое значение	Как измерять
Время до первого черновика	< 15 минут	От запуска команды до получения итогового_черновика/
Охват доказательств	> 80%	Доля утверждений с уверенностью «высокая» или «средняя»
Критические пробелы	< 3 на функцию	Количество записей в разделе «критические» пробелы.json
Объём ручных правок	< 30% текста	Разница между черновиком и финальным коммитом (git diff)

Качественные критерии

1. **Полезность для нового сотрудника.** Дать черновик коллеге, незнакомому с функцией. Вопрос: «Помогло ли разобраться за час?»
2. **Корректность.** Сколько фактических ошибок (ложных утверждений) в документации? Цель: ноль. Пробелы допустимы, ошибки — нет.
3. **Экономия времени.** Сколько времени ушло бы на ручное написание аналогичной документации? Ориентир: 1–2 дня вручную против нескольких часов с системой.

Риски

1. **Домысливание.** Агент может зафиксировать поведение, которого нет в коде. Агент проверки доказательств должен это ловить. После пилота — сравнить черновик с реальным кодом, подсчитать ложные утверждения.
2. **Переполнение контекста.** Если функция крупнее ожидаемого, система должна предупредить и предложить разбить. Это нужно протестировать в пилоте явно.
3. **Несогласованная терминология.** В коде — «account», в Jira — «счёт», в Confluence — «лицевой счёт». Агент ревью должен выявлять такие расхождения через словарь терминов.

Рекомендуемый объём пилота

- 2–3 функции средней сложности
- 5 агентов, 1 свод правил, 2 контрольных блока
- Длительность: 2–3 недели (включая настройку и сбор обратной связи)
- Участники: 2–3 аналитика, 1 инженер для технической поддержки

После пилота: замерить показатели, собрать обратную связь, принять решение — масштабировать или пересматривать архитектуру.

Заключение

GigaCode CLI для обратного анализа — это не замена аналитика, а инструмент для систематизации разбора кода. Система берёт на себя механическую работу: поиск файлов, построение графа зависимостей, извлечение контекста из Jira и Confluence, подготовку черновика и проверку доказательств. Аналитик остаётся ответственным за критические решения и за финальное утверждение результата. Иначе говоря, цель решения — не “автоматизировать аналитику целиком”, а автоматизировать сбор доказательств, построение карты реализации и подготовку проверяемого черновика. Спорные места, противоречия и границы функции сознательно оставляются под контролем человека.

Ключевые преимущества

- **Доказуемость:** каждое утверждение имеет источник, неподтверждённое уходит в предположения
- **Воспроизводимость:** любой сотрудник запускает анализ по тому же алгоритму
- **Документация как код:** хранится в репозитории, проходит проверку, история версий сохраняется
- **Стратегия разбивки:** система работает с большими кодовыми базами через проекции и фильтрацию
- **Контроль качества:** при низком охвате доказательств система возвращается на картирование, а не выдаёт мусор

Следующие шаги

Настроить общий skill reverse-analysis и два обязательных hook-a: preflight и output validation

1. Выбрать 2–3 функции для пилота
2. Настроить инфраструктуру: коннектор Atlassian, repomix, graphify
3. Написать шаблон_функции.md и контракты_агентов.md
4. Провести пилот с 2–3 аналитиками
5. Замерить показатели и собрать обратную связь
6. Принять решение о масштабировании